

DAY 3: Videos and watch progress, BACKEND

Admins and instructors upload lesson videos; customers who bought the course watch them; the staff see how far customers got. **12 new files + 3 edits.**

Needs Day 2 first (`InstructorService`, `InstructorCourseService`, `CourseDeleteListener`).

1. "Store the videos in the database": the decision

You confirmed: **details in the database, the file on disk.** Here is why that is the standard way.

- A video is large (tens or hundreds of MB). Stored as a `LONGLOB` inside MySQL it would bloat backups, hit `max_allowed_packet` limits, slow every query on that table, and make seeking inside a video very hard.
- So the **database row** (`course_videos`) keeps the *details*: which course, title, description, size, length, who uploaded, when, and the random file name. The **bytes** live in a folder (`video.storage-dir`, default `video-files`).
- Back up the database **and** that folder together.

2. How it works, from zero

Upload. The browser sends the file as `multipart/form-data`. `VideoStorageService.store` does four checks before it keeps anything:

1. the declared size is under `video.max-size-mb` (default 200);
2. the **first bytes of the file** decide the type: MP4 has `ftyp` at byte 4, WebM starts with `1A 45 DF A3`. A virus renamed `lesson.mp4` is refused. The file name the user typed is never trusted;
3. while copying, the size is counted again, and a file that grows past the limit is deleted at once;
4. the file gets a random name (`UUID.mp4`), so nobody can guess it or overwrite another file, and `../` tricks are impossible (`resolve` only accepts names matching the exact pattern).

Why a "ticket" for watching. An HTML `<video src="...">` cannot send the `Authorization: Bearer` header. So:

1. The page asks `GET /api/video/{id}/ticket` **with the normal login**. The server checks the access rules and returns a signed ticket (HMAC-SHA256 using your `jwt.secret`) that names **exactly one video** and expires (default 120 minutes).
2. The page sets `<video src=".../api/video/stream/{id}?t=TICKET">`.
3. The stream endpoint is open in Security but refuses any request whose ticket is wrong, expired, edited or for another video.

Why Range requests. When a viewer drags the slider to minute 20, the browser sends `Range: bytes=52428800-`. `RangeParser` reads that header and `VideoStreamer` seeks to that byte and sends HTTP **206 Partial Content**. Without this, seeking would not work and the whole file would download first.

Who may do what (VideoService)

Action	ADMIN	INSTRUCTOR	CUSTOMER
upload, delete, see stats	any course	only own courses, and only if APPROVED	no
list and watch	any course	own courses	only courses they bought
save watch progress	no	no	yes (own rows only)
my learning	no	no	yes

Progress. One row per customer and video (video_progress, unique pair). The page sends the position every 10 seconds, on pause, on switching lesson, and on leaving. The server keeps the furthest point reached, and marks the video **completed at 90%**. Going back later never un-completes it. The server clamps silly positions (negative, past the end).

Staff stats. For each video: how many customers started it, how many finished, and the average percent. Plus how many customers are enrolled.

Cleanup. Deleting a video removes the row, the file and everyone's progress. When an instructor deletes a course, VideoService (a CourseDeleteListener) removes all its videos too.

3. Honest status

Compiled with real Java against stand-in Spring classes and run with fake repositories and **real temporary files: 101 checks, 0 failures**. They cover the Range parser, signed tickets (valid, wrong video, expired, tampered, other secret), magic-byte checks, size limits with no leftover files, the path trick, real byte-for-byte full and partial streaming, the whole access matrix, upload rules, progress clamping and completion, stats, my-learning and every delete path. The browser side was tested with a **real playable WebM (48 checks**, see the frontend guide).

Not tested: Spring start-up, MySQL, real multipart handling, real HTTP security and real MP4 playback (the browser test used WebM). Compile errors are possible; send me the text.

4. Edits to existing files

4a. config/SecurityConfig.java: add these rules **above** `.anyRequest().authenticated()`, in this order (first match wins):

```
// videos (Day 3)
.requestMatchers(HttpMethod.GET, "/api/video/stream/*").permitAll()           // the ticket is checked
inside
.requestMatchers(HttpMethod.GET, "/api/video/my-learning").hasAuthority("CUSTOMER")
.requestMatchers(HttpMethod.PUT, "/api/video/*/progress").hasAuthority("CUSTOMER")
.requestMatchers("/api/video/**").hasAnyAuthority("ADMIN", "INSTRUCTOR", "CUSTOMER")
```

4b. src/main/resources/application.properties: append the first three lines below. Then **change** (do not duplicate) the two `spring.servlet.multipart` lines that CHANGES-7 already added (`max-file-size=20MB`, `max-request-size=21MB`) to the larger values shown:

```
# ---- Videos (Day 3) ----
video.storage-dir=${VIDEO_DIR:video-files}
video.max-size-mb=${VIDEO_MAX_MB:200}
video.ticket-minutes=${VIDEO_TICKET_MINUTES:120}

# CHANGE the two existing lines to these. Spring itself must accept the big upload, so they must be at
# least video.max-size-mb.
spring.servlet.multipart.max-file-size=200MB
spring.servlet.multipart.max-request-size=201MB
```

The bigger multipart limit also applies to the image upload, but `UploadController` checks its own 5 MB rule, so nothing changes for images.

4c. `config/RateLimitFilter.java`: your global limit is 120 requests per minute per IP. A viewer who scrubs through a video makes many Range requests, so let the stream endpoint skip that counter (it is protected by the ticket instead). In `doFilterInternal`, directly after `String path = request.getRequestURI();` add:

```
if (path.startsWith("/api/video/stream/")) {
    chain.doFilter(request, response);
    return;
}
```

Add `video-files/` to `.gitignore` so video files never go to GitHub.

Why not inside `uploads/`? CHANGES-7 serves everything under `uploads/` publicly (so course pictures show in `<img>` tags). Videos must **not** be reachable that way, so they go in a separate folder, `video-files`, that no URL points to. Only the ticket-checked stream endpoint can read it.

5. Check it

1. Admin (Swagger or curl with the admin token): `POST /api/video/course/1` as form-data with `title` and `file` (a small `.mp4`). Answer 201.
2. Upload a `.txt` renamed to `.mp4`: **400 "Only MP4 or WebM videos can be uploaded"**, and no new file appears in `video-files`.
3. Customer who **bought** course 1: `GET /api/video/course/1` lists it; `GET /api/video/{id}/ticket` returns a `url`. Opening `http://localhost:8080` + that url in the browser plays the video. Change one character of the ticket: **403**.
4. A customer who did **not** buy the course: 403 "Enroll in this course to watch its videos".
5. `PUT /api/video/{id}/progress` with `{"positionSec": 95, "durationSec": 100}`: `completed: true`.
6. Admin or the owning instructor: `GET /api/video/course/1/stats` shows viewers, finished and average.
7. An instructor uploading to someone else's course: 403.

6. Troubleshooting

Symptom	Cause
413 or "larger than the upload limit" for a video	the two <code>spring.servlet.multipart</code> lines are still at 20MB/21MB (step 4b)
500 "could not be saved"	the app cannot write to <code>video.storage-dir</code> ; check folder permissions
the video does not seek	a proxy removed the <code>Range</code> header, or you served the file another way
429 while scrubbing	edit 4c missed
403 on the stream URL	the ticket expired (it refreshes automatically once in the page) or the URL was edited
video will not play but the download works	the browser cannot decode that MP4 codec; export it as H.264 + AAC

7. Complete code

`model/CourseVideo.java` (new file)

```
package com.examly.springapp.model;

import jakarta.persistence.*;

/**
 * Details (metadata) of one uploaded lesson video. The video file itself lives on disk under
 * {@code video.storage-dir}; this row says what it is and where it belongs.
 * The course is kept as a plain number on purpose, so the existing Course table is not changed.
 */
@Entity
@Table(name = "course_videos", indexes = @Index(name = "idx_course_videos_course", columnList = "courseId"))
public class CourseVideo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long courseId;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(length = 500)
    private String description;

    /** Random name on disk, never the name the user chose. */
    @Column(nullable = false, unique = true, length = 80)
    private String storedName;

    @Column(length = 200)
    private String originalName;

    @Column(nullable = false, length = 50)
    private String contentType;

    @Column(nullable = false)
    private long sizeBytes;

    /** Length in seconds as reported by the uploader's browser; 0 when unknown. */
    @Column(nullable = false)
    private int durationSec;

    @Column(nullable = false)
    private Long uploadedByUserId;

    @Column(nullable = false)
```

```

    private long uploadedAtMs;

    public CourseVideo() {}

    public CourseVideo(Long courseId, String title, String description, String storedName, String
originalName,
                        String contentType, long sizeBytes, int durationSec, Long uploadedByUserId, long
uploadedAtMs) {
        this.courseId = courseId;
        this.title = title;
        this.description = description;
        this.storedName = storedName;
        this.originalName = originalName;
        this.contentType = contentType;
        this.sizeBytes = sizeBytes;
        this.durationSec = durationSec;
        this.uploadedByUserId = uploadedByUserId;
        this.uploadedAtMs = uploadedAtMs;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Long getCourseId() { return courseId; }
    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public String getStoredName() { return storedName; }
    public String getOriginalName() { return originalName; }
    public String getContentType() { return contentType; }
    public long getSizeBytes() { return sizeBytes; }
    public int getDurationSec() { return durationSec; }
    public Long getUploadedByUserId() { return uploadedByUserId; }
    public long getUploadedAtMs() { return uploadedAtMs; }
}

```

model/VideoProgress.java (new file)

```

package com.examly.springapp.model;

import jakarta.persistence.*;

/** How far one customer has watched one video (one row per customer and video). */
@Entity
@Table(name = "video_progress",
        uniqueConstraints = @UniqueConstraint(name = "uk_video_progress", columnNames = {"customerId",
"videoId"}))
public class VideoProgress {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long customerId;

    @Column(nullable = false)
    private Long videoId;

    /** Where the customer stopped, in seconds. */
    @Column(nullable = false)
    private int positionSec;

    /** The furthest point reached, in seconds. Completion is judged on this so skipping back does not undo
it. */
    @Column(nullable = false)
    private int maxPositionSec;

    @Column(nullable = false)
    private boolean completed;

    @Column(nullable = false)

```

```

private long updatedAtMs;

public VideoProgress() {}

public VideoProgress(Long customerId, Long videoId) {
    this.customerId = customerId;
    this.videoId = videoId;
}

public Long getId() { return id; }
public Long getCustomerId() { return customerId; }
public Long getVideoId() { return videoId; }
public int getPositionSec() { return positionSec; }
public void setPositionSec(int positionSec) { this.positionSec = positionSec; }
public int getMaxPositionSec() { return maxPositionSec; }
public void setMaxPositionSec(int maxPositionSec) { this.maxPositionSec = maxPositionSec; }
public boolean isCompleted() { return completed; }
public void setCompleted(boolean completed) { this.completed = completed; }
public long getUpdatedAtMs() { return updatedAtMs; }
public void setUpdatedAtMs(long updatedAtMs) { this.updatedAtMs = updatedAtMs; }
}

```

repository/CourseVideoRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.CourseVideo;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for video details. */
@Repository
public interface CourseVideoRepo extends JpaRepository<CourseVideo, Long> {
    List<CourseVideo> findByCourseIdOrderByIdAsc(Long courseId);
    List<CourseVideo> findByCourseIdIn(List<Long> courseIds);
    long countByCourseId(Long courseId);
}

```

repository/VideoProgressRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.VideoProgress;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for watch progress. */
@Repository
public interface VideoProgressRepo extends JpaRepository<VideoProgress, Long> {
    Optional<VideoProgress> findByCustomerIdAndVideoId(Long customerId, Long videoId);
    List<VideoProgress> findByCustomerIdAndVideoIdIn(Long customerId, List<Long> videoIds);
    List<VideoProgress> findByVideoIdIn(List<Long> videoIds);
    void deleteByVideoId(Long videoId);
}

```

dto/VideoDtos.java (new file)

```

package com.examly.springapp.dto;

import java.util.List;

/** Shapes of the video API. */

```

```

public final class VideoDtos {
    private VideoDtos() {}

    /** One video in a course. {@code progress} is filled only for a customer. */
    public static class VideoView {
        public Long id;
        public Long courseId;
        public String title;
        public String description;
        public long sizeBytes;
        public int durationSec;
        public long uploadedAtMs;
        public ProgressView progress;
    }

    public static class ProgressView {
        public int positionSec;
        public int percent;
        public boolean completed;
    }

    public static class ProgressRequest {
        public Integer positionSec;
        public Integer durationSec;
    }

    /** A short-lived pass that lets the browser's video player fetch one video. */
    public static class TicketView {
        public String ticket;
        public String url;
        public int expiresInSeconds;
    }

    /** What the staff see for one video: how many customers started and finished it. */
    public static class VideoStat {
        public Long videoId;
        public String title;
        public int durationSec;
        public int viewers;
        public int completedCount;
        public int averagePercent;
    }

    public static class CourseStats {
        public Long courseId;
        public int enrolledCustomers;
        public List<VideoStat> videos;
    }

    /** One course in a customer's "my learning" page. */
    public static class LearningItem {
        public Long courseId;
        public String courseType;
        public int totalVideos;
        public int completedVideos;
        public int percent;
        public Long resumeVideoId;
        public int resumePositionSec;
    }
}

```

video/RangeParser.java (new file)

```

package com.examly.springapp.video;

/**
 * Reads the HTTP "Range" header a video player sends, for example "bytes=1000-" when the viewer
 * jumps ahead in the video. Only one range is supported, which is all browsers ask for.
 */

```

```

public final class RangeParser {
    private RangeParser() {}

    /** First and last byte to send (both inclusive). */
    public static final class Range {
        public final long start;
        public final long end;
        public Range(long start, long end) { this.start = start; this.end = end; }
        public long length() { return end - start + 1; }
    }

    /** Thrown when the asked bytes are outside the file; the answer is then HTTP 416. */
    public static class NotSatisfiableException extends RuntimeException {
        public NotSatisfiableException(String message) { super(message); }
    }

    /**
     * @return the range to send, or {@code null} when the whole file should be sent
     *         (no header, a header we do not understand, or more than one range)
     */
    public static Range parse(String header, long fileLength) {
        if (header == null || fileLength <= 0) return null;
        String h = header.trim();
        if (!h.regionMatches(true, 0, "bytes=", 0, 6)) return null;
        String spec = h.substring(6).trim();
        if (spec.isEmpty() || spec.contains(",")) return null;
        int dash = spec.indexOf('-');
        if (dash < 0) return null;
        String a = spec.substring(0, dash).trim();
        String b = spec.substring(dash + 1).trim();
        try {
            if (a.isEmpty()) {
                // "-500" means the last 500 bytes
                if (b.isEmpty()) return null;
                long n = Long.parseLong(b);
                if (n <= 0) throw new NotSatisfiableException("Range not satisfiable");
                long start = Math.max(0, fileLength - n);
                return new Range(start, fileLength - 1);
            }
            long start = Long.parseLong(a);
            long end = b.isEmpty() ? fileLength - 1 : Long.parseLong(b);
            if (start < 0 || start >= fileLength || end < start) throw new NotSatisfiableException("Range
not satisfiable");
            return new Range(start, Math.min(end, fileLength - 1));
        } catch (NumberFormatException e) {
            return null;
        }
    }
}

```

video/VideoStreamer.java (new file)

```

package com.examly.springapp.video;

import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.OutputStream;
import java.io.RandomAccessFile;
import java.nio.file.Path;

/** Writes a video file to the response, whole or just the part the player asked for. */
public final class VideoStreamer {
    private static final int BUFFER = 64 * 1024;

    private VideoStreamer() {}

    /**
     * @param rangeHeader the Range header of the request, may be null
     * @throws RangeParser.NotSatisfiableException when the range is outside the file (the caller answers

```

```

416)
    */
    public static void write(Path file, String contentType, String rangeHeader, HttpServletResponse
response) throws IOException {
        long length = file.toFile().length();
        RangeParser.Range range = RangeParser.parse(rangeHeader, length);
        response.setContentType(contentType);
        response.setHeader("Accept-Ranges", "bytes");
        response.setHeader("Cache-Control", "private, max-age=0, no-store");
        response.setHeader("X-Content-Type-Options", "nosniff");
        long start = 0;
        long count = length;
        if (range == null) {
            response.setStatus(200);
        } else {
            start = range.start;
            count = range.length();
            response.setStatus(206);
            response.setHeader("Content-Range", "bytes " + range.start + "-" + range.end + "/" + length);
        }
        response.setContentLengthLong(count);
        try (RandomAccessFile in = new RandomAccessFile(file.toFile(), "r")) {
            in.seek(start);
            OutputStream out = response.getOutputStream();
            byte[] buf = new byte[BUFFER];
            long left = count;
            while (left > 0) {
                int n = in.read(buf, 0, (int) Math.min(buf.length, left));
                if (n < 0) break;
                out.write(buf, 0, n);
                left -= n;
            }
            out.flush();
        }
    }
}

```

video/VideoTicketService.java (new file)

```

package com.examly.springapp.video;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Base64;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

/**
 * A browser's video player cannot send the Authorization header, so the page first asks for a short-lived
 * "ticket" (with the normal login) and puts it in the video address. The ticket is signed with the server
 * secret, names exactly one video and expires, so a copied address stops working after a while.
 */
@Service
public class VideoTicketService {
    private final byte[] key;
    private final long ttlMs;

    public VideoTicketService(@Value("${jwt.secret}") String secret,
                              @Value("${video.ticket-minutes:120}") long ticketMinutes) {
        if (secret == null || secret.length() < 32) throw new IllegalStateException("jwt.secret must be at
least 32 characters long");
        this.key = ("video-ticket:" + secret).getBytes(StandardCharsets.UTF_8);
        this.ttlMs = Math.max(1, ticketMinutes) * 60_000L;
    }

    public long ttlSeconds() { return ttlMs / 1000; }
}

```

```

    public String issue(Long videoId, Long userId) { return issue(videoId, userId,
System.currentTimeMillis()); }

    public String issue(Long videoId, Long userId, long nowMs) {
        String payload = videoId + ":" + userId + ":" + (nowMs + ttlMs);
        String body =
Base64.getEncoder().withoutPadding().encodeToString(payload.getBytes(StandardCharsets.UTF_8));
        return body + "." + sign(body);
    }

    public boolean isValid(String ticket, Long videoId) { return isValid(ticket, videoId,
System.currentTimeMillis()); }

    public boolean isValid(String ticket, Long videoId, long nowMs) {
        try {
            if (ticket == null || videoId == null) return false;
            int dot = ticket.indexOf('.');
            if (dot <= 0 || dot == ticket.length() - 1) return false;
            String body = ticket.substring(0, dot);
            String sig = ticket.substring(dot + 1);
            if (!MessageDigest.isEqual(sign(body).getBytes(StandardCharsets.UTF_8),
sig.getBytes(StandardCharsets.UTF_8))) return false;
            String[] parts = new String(Base64.getDecoder().decode(body),
StandardCharsets.UTF_8).split(":");
            if (parts.length != 3) return false;
            return Long.parseLong(parts[0]) == videoId && Long.parseLong(parts[2]) > nowMs;
        } catch (RuntimeException e) {
            return false;
        }
    }

    private String sign(String body) {
        try {
            Mac mac = Mac.getInstance("HmacSHA256");
            mac.init(new SecretKeySpec(key, "HmacSHA256"));
            return
Base64.getEncoder().withoutPadding().encodeToString(mac.doFinal(body.getBytes(StandardCharsets.UTF_8)));
        } catch (Exception e) {
            throw new IllegalStateException("Could not sign the video ticket", e);
        }
    }
}

```

video/VideoStorageService.java (new file)

```

package com.examly.springapp.video;

import com.examly.springapp.exception.InvalidRequestException;
import java.io.BufferedInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.UUID;
import java.util.regex.Pattern;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

/**
 * Keeps the video files on disk. The type is decided from the first bytes of the file (not from its name),
 * the size is limited while copying, and the file gets a random name so nobody can guess or overwrite it.
 */
@Service
public class VideoStorageService {
    private static final Logger log = LoggerFactory.getLogger(VideoStorageService.class);
}

```

```

private static final Pattern STORED_NAME = Pattern.compile("[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}\\.(mp4|webm)$");

/** The result of saving a file. */
public static final class Stored {
    public final String storedName;
    public final String contentType;
    public final long sizeBytes;
    public Stored(String storedName, String contentType, long sizeBytes) {
        this.storedName = storedName;
        this.contentType = contentType;
        this.sizeBytes = sizeBytes;
    }
}

private final Path root;
private final long maxBytes;

public VideoStorageService(@Value("${video.storage-dir:video-files}") String dir,
                           @Value("${video.max-size-mb:200}") long maxMb) {
    this.root = Paths.get(dir).toAbsolutePath().normalize();
    this.maxBytes = Math.max(1, maxMb) * 1024L * 1024L;
}

public long maxBytes() { return maxBytes; }

/** "mp4", "webm", or null when the first bytes are not a supported video. */
public static String detectExtension(byte[] head, int n) {
    if (n >= 12 && head[4] == 'f' && head[5] == 't' && head[6] == 'y' && head[7] == 'p') return "mp4";
    if (n >= 4 && (head[0] & 0xFF) == 0x1A && (head[1] & 0xFF) == 0x45 && (head[2] & 0xFF) == 0xDF &&
(head[3] & 0xFF) == 0xA3) return "webm";
    return null;
}

public Stored store(InputStream raw, long declaredSize) {
    if (declaredSize > maxBytes) throw tooBig();
    try (InputStream in = new BufferedInputStream(raw)) {
        in.mark(16);
        byte[] head = new byte[16];
        int n = in.readNBytes(head, 0, head.length);
        String ext = detectExtension(head, n);
        if (ext == null) throw new InvalidRequestException("Only MP4 or WebM videos can be uploaded");
        in.reset();
        Files.createDirectories(root);
        String name = UUID.randomUUID() + "." + ext;
        Path target = root.resolve(name).normalize();
        long total = 0;
        try (OutputStream out = Files.newOutputStream(target)) {
            byte[] buf = new byte[64 * 1024];
            int r;
            while ((r = in.read(buf)) > 0) {
                total += r;
                if (total > maxBytes) {
                    out.close();
                    Files.deleteIfExists(target);
                    throw tooBig();
                }
                out.write(buf, 0, r);
            }
        } catch (IOException | RuntimeException e) {
            Files.deleteIfExists(target);
            throw e;
        }
        return new Stored(name, "mp4".equals(ext) ? "video/mp4" : "video/webm", total);
    } catch (IOException e) {
        log.error("Could not save the video", e);
        throw new IllegalStateException("The video could not be saved. Please try again.");
    }
}

/** Full path of a stored file, or null if the name is not one we created (blocks "../" tricks). */

```

```

    public Path resolve(String storedName) {
        if (storedName == null || !STORED_NAME.matcher(storedName).matches()) return null;
        Path p = root.resolve(storedName).normalize();
        return p.startsWith(root) && Files.isRegularFile(p) ? p : null;
    }

    /** Removes a file; a missing file is fine. Never throws, so a failure cannot block a database delete.
    */
    public void delete(String storedName) {
        try {
            if (storedName != null && STORED_NAME.matcher(storedName).matches())
                Files.deleteIfExists(root.resolve(storedName).normalize());
        } catch (IOException e) {
            log.warn("Could not remove the video file {}", storedName);
        }
    }

    private InvalidRequestException tooBig() {
        return new InvalidRequestException("The video is larger than " + (maxBytes / 1024 / 1024) + " MB");
    }
}

```

service/VideoService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.VideoDtos.*;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.CourseVideo;
import com.examly.springapp.model.Orders;
import com.examly.springapp.model.User;
import com.examly.springapp.model.VideoProgress;
import com.examly.springapp.repository.CourseRepo;
import com.examly.springapp.repository.CourseVideoRepo;
import com.examly.springapp.repository.OrderRepo;
import com.examly.springapp.repository.VideoProgressRepo;
import com.examly.springapp.video.VideoStorageService;
import com.examly.springapp.video.VideoTicketService;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

/**
 * Lesson videos: who may upload, who may watch, and how far each customer has watched.
 * <ul>
 * <li>ADMIN: any course.</li>
 * <li>INSTRUCTOR: only their own courses, and only once approved.</li>
 * <li>CUSTOMER: only courses they bought; they also save their own progress.</li>
 * </ul>
 */
@Service
public class VideoService implements CourseDeleteListener {
    private static final Logger log = LoggerFactory.getLogger(VideoService.class);
    /** A video counts as finished when the viewer reached this share of it. */
    static final int COMPLETE_PERCENT = 90;
}

```

```

private static final int MAX_SECONDS = 24 * 60 * 60;

private final AccessService access;
private final CourseRepo courses;
private final CourseVideoRepo videos;
private final VideoProgressRepo progressRepo;
private final OrderRepo orders;
private final InstructorService instructors;
private final InstructorCourseService instructorCourses;
private final VideoStorageService storage;
private final VideoTicketService tickets;

@Autowired
public VideoService(AccessService access, CourseRepo courses, CourseVideoRepo videos, VideoProgressRepo
progressRepo,
                    OrderRepo orders, InstructorService instructors, InstructorCourseService
instructorCourses,
                    VideoStorageService storage, VideoTicketService tickets) {
    this.access = access;
    this.courses = courses;
    this.videos = videos;
    this.progressRepo = progressRepo;
    this.orders = orders;
    this.instructors = instructors;
    this.instructorCourses = instructorCourses;
    this.storage = storage;
    this.tickets = tickets;
}

// ----- access rules

private void requireCourse(Long courseId) {
    if (courseId == null || courseId <= 0 || !DatabaseOperationException.guard("checking the course",
() -> courses.existsById(courseId))) {
        throw new ResourceNotFoundException("Course not found");
    }
}

/** Throws unless the caller is the admin or the approved instructor who owns the course. */
private User requireStaffFor(Long courseId) {
    requireCourse(courseId);
    User u = access.currentUser();
    if (access.isAdmin()) return u;
    User approved = instructors.requireApproved(access.currentEmail());
    if (!instructorCourses.owns(approved, courseId)) throw new AccessDeniedException("You can only
manage videos of your own courses");
    return approved;
}

private boolean isStaff() {
    if (access.isAdmin()) return true;
    return "INSTRUCTOR".equals(access.currentUser().getRole());
}

private Set<Long> boughtCourseIds(Long customerId) {
    Set<Long> ids = new LinkedHashSet<>();
    List<Orders> list = DatabaseOperationException.guard("loading your enrollments", () ->
orders.findByCustomer_CustomerId(customerId));
    for (Orders o : list) for (Course c : o.getCourses()) ids.add(c.getCourseId());
    return ids;
}

/** Allows staff who manage the course, or a customer who bought it. Returns the customer id (null for
staff). */
private Long requireViewer(Long courseId) {
    requireCourse(courseId);
    if (isStaff()) {
        requireStaffFor(courseId);
        return null;
    }
    Long customerId = access.currentCustomer().getCustomerId();

```

```

        if (!boughtCourseIds(customerId).contains(courseId)) throw new AccessDeniedException("Enroll in
this course to watch its videos");
        return customerId;
    }

    private CourseVideo videoOrThrow(Long videoId) {
        if (videoId == null || videoId <= 0) throw new ResourceNotFoundException("Video not found");
        return DatabaseOperationException.guard("loading the video", () -> videos.findById(videoId))
            .orElseThrow(() -> new ResourceNotFoundException("Video not found"));
    }

    // ----- staff: upload, delete

    public VideoView upload(Long courseId, String titleRaw, String descriptionRaw, Integer durationSec,
MultipartFile file) {
        User u = requireStaffFor(courseId);
        String title = titleRaw == null ? "" : titleRaw.trim();
        if (title.length() < 2 || title.length() > 150) throw new InvalidRequestException("Title must be 2
to 150 characters");
        String description = descriptionRaw == null ? "" : descriptionRaw.trim();
        if (description.length() > 500) throw new InvalidRequestException("Description can have at most 500
characters");
        if (file == null || file.isEmpty()) throw new InvalidRequestException("Choose a video file to
upload");
        int duration = durationSec == null ? 0 : Math.max(0, Math.min(MAX_SECONDS, durationSec));

        VideoStorageService.Stored stored;
        try {
            stored = storage.store(file.getInputStream(), file.getSize());
        } catch (java.io.IOException e) {
            log.error("Could not read the uploaded video", e);
            throw new InvalidRequestException("The upload could not be read. Please try again.");
        }
        String original = file.getOriginalFilename();
        if (original != null && original.length() > 200) original = original.substring(0, 200);
        CourseVideo v = new CourseVideo(courseId, title, description.isEmpty() ? null : description,
stored.storedName, original,
            stored.contentType, stored.sizeBytes, duration, u.getUserId(), System.currentTimeMillis());
        try {
            return toView(DatabaseOperationException.guard("saving the video", () -> videos.save(v)),
null);
        } catch (RuntimeException e) {
            storage.delete(stored.storedName); // do not leave a file with no row
            throw e;
        }
    }

    public void delete(Long videoId) {
        CourseVideo v = videoOrThrow(videoId);
        requireStaffFor(v.getCourseId());
        removeVideo(v);
    }

    private void removeVideo(CourseVideo v) {
        DatabaseOperationException.guardVoid("removing the watch progress", () ->
progressRepo.deleteByVideoId(v.getId()));
        DatabaseOperationException.guardVoid("removing the video", () -> videos.delete(v));
        storage.delete(v.getStoredName());
    }

    /** Called when an instructor deletes a course: its videos and their progress go too. Never blocks the
delete. */
    @Override
    public void onCourseDeleted(Long courseId) {
        try {
            for (CourseVideo v : videos.findByCourseIdOrderByIdAsc(courseId)) removeVideo(v);
        } catch (RuntimeException e) {
            log.warn("Could not remove all videos of course {}: {}", courseId, e.getMessage());
        }
    }
}

```

```

// ----- list, ticket, stream

public List<VideoView> list(Long courseId) {
    Long customerId = requireViewer(courseId);
    List<CourseVideo> rows = DatabaseOperationException.guard("loading the videos", () ->
videos.findByCourseIdOrderByIdAsc(courseId));
    Map<Long, VideoProgress> mine = new HashMap<>();
    if (customerId != null && !rows.isEmpty()) {
        List<Long> ids = rows.stream().map(CourseVideo::getId).toList();
        for (VideoProgress p : progressRepo.findByCustomerIdAndVideoIdIn(customerId, ids))
mine.put(p.getId(), p);
    }
    List<VideoView> out = new ArrayList<>();
    for (CourseVideo v : rows) out.add(toView(v, mine.get(v.getId())));
    return out;
}

public TicketView ticket(Long videoId) {
    CourseVideo v = videoOrThrow(videoId);
    requireViewer(v.getCustomerId());
    TicketView t = new TicketView();
    t.ticket = tickets.issue(v.getId(), access.currentUser().getUserId());
    t.url = "/api/video/stream/" + v.getId() + "?t=" + t.ticket;
    t.expiresInSeconds = (int) tickets.ttlSeconds();
    return t;
}

/** For the stream endpoint: no login header is possible there, so the ticket is the proof. */
public CourseVideo videoForStream(Long videoId, String ticket) {
    if (!tickets.isValid(ticket, videoId)) throw new AccessDeniedException("This video link has
expired. Open the lesson again.");
    return videoOrThrow(videoId);
}

// ----- progress

public ProgressView saveProgress(Long videoId, ProgressRequest req) {
    if (req == null || req.positionSec == null) throw new InvalidRequestException("positionSec is
required");
    CourseVideo v = videoOrThrow(videoId);
    Long customerId = requireCustomerViewer(v.getCustomerId());
    int duration = v.getDurationSec() > 0 ? v.getDurationSec()
        : (req.durationSec == null ? 0 : Math.max(0, Math.min(MAX_SECONDS, req.durationSec)));
    int pos = Math.max(0, Math.min(req.positionSec, duration > 0 ? duration : MAX_SECONDS));

    VideoProgress p = progressRepo.findByCustomerIdAndVideoId(customerId, videoId).orElseGet(() -> new
VideoProgress(customerId, videoId));
    p.setPositionSec(pos);
    p.setMaxPositionSec(Math.max(p.getMaxPositionSec(), pos));
    if (!p.isCompleted() && duration > 0 && (long) p.getMaxPositionSec() * 100 >= (long) duration *
COMPLETE_PERCENT) p.setCompleted(true);
    p.setUpdatedAtMs(System.currentTimeMillis());
    VideoProgress saved = DatabaseOperationException.guard("saving your progress", () ->
progressRepo.save(p));
    return toProgress(saved, duration);
}

private Long requireCustomerViewer(Long courseId) {
    Long customerId = requireViewer(courseId);
    if (customerId == null) throw new AccessDeniedException("Only customers have watch progress");
    return customerId;
}

// ----- staff: progress of the customers

public CourseStats stats(Long courseId) {
    requireStaffFor(courseId);
    List<CourseVideo> rows = videos.findByCourseIdOrderByIdAsc(courseId);
    Set<Long> enrolled = new HashSet<>();
    for (Orders o : DatabaseOperationException.guard("loading the enrollments", () ->
orders.findByCourses_CourseId(courseId))) {

```

```

        if (o.getCustomer() != null) enrolled.add(o.getCustomer().getCustomerId());
    }
    CourseStats s = new CourseStats();
    s.courseId = courseId;
    s.enrolledCustomers = enrolled.size();
    s.videos = new ArrayList<>();
    Map<Long, List<VideoProgress>> byVideo = new HashMap<>();
    if (!rows.isEmpty()) {
        for (VideoProgress p :
progressRepo.findByVideoIdIn(rows.stream().map(CourseVideo::getId).toList())) {
            byVideo.computeIfAbsent(p.getVideoId(), k -> new ArrayList<>()).add(p);
        }
    }
    for (CourseVideo v : rows) {
        VideoStat st = new VideoStat();
        st.videoId = v.getId();
        st.title = v.getTitle();
        st.durationSec = v.getDurationSec();
        int viewers = 0, done = 0, sum = 0;
        for (VideoProgress p : byVideo.getOrDefault(v.getId(), List.of())) {
            if (p.getMaxPositionSec() <= 0 && !p.isCompleted()) continue;
            viewers++;
            if (p.isCompleted()) done++;
            sum += percentOf(p, v.getDurationSec());
        }
        st.viewers = viewers;
        st.completedCount = done;
        st.averagePercent = viewers == 0 ? 0 : sum / viewers;
        s.videos.add(st);
    }
    return s;
}

// ----- customer: my learning

public List<LearningItem> myLearning() {
    Long customerId = access.currentCustomer().getCustomerId();
    Set<Long> courseIds = boughtCourseIds(customerId);
    List<LearningItem> out = new ArrayList<>();
    if (courseIds.isEmpty()) return out;
    List<CourseVideo> all = videos.findByCourseIdIn(new ArrayList<>(courseIds));
    if (all.isEmpty()) return out;
    Map<Long, VideoProgress> mine = new HashMap<>();
    for (VideoProgress p : progressRepo.findByIdAndVideoIdIn(customerId,
all.stream().map(CourseVideo::getId).toList())) mine.put(p.getVideoId(), p);
    Map<Long, Course> byId = new HashMap<>();
    for (Course c : courses.findAllById(courseIds)) byId.put(c.getCourseId(), c);

    for (Long courseId : courseIds) {
        List<CourseVideo> list = all.stream().filter(v ->
v.getCourseId().equals(courseId)).sorted(Comparator.comparing(CourseVideo::getId)).toList();
        if (list.isEmpty() || !byId.containsKey(courseId)) continue;
        LearningItem it = new LearningItem();
        it.courseId = courseId;
        it.courseType = byId.get(courseId).getCourseType();
        it.totalVideos = list.size();
        long newest = -1;
        for (CourseVideo v : list) {
            VideoProgress p = mine.get(v.getId());
            if (p != null && p.isCompleted()) { it.completedVideos++; continue; }
            if (p != null && p.getUpdatedAtMs() > newest) { newest = p.getUpdatedAtMs(); }
it.resumeVideoId = v.getId(); it.resumePositionSec = p.getPositionSec(); }
        }
        if (it.resumeVideoId == null) {
            for (CourseVideo v : list) {
                VideoProgress p = mine.get(v.getId());
                if (p == null || !p.isCompleted()) { it.resumeVideoId = v.getId(); it.resumePositionSec
= 0; break; }
            }
        }
        it.percent = it.completedVideos * 100 / it.totalVideos;
    }
}

```

```

        out.add(it);
    }
    return out;
}

// ----- mapping

private static int percentOf(VideoProgress p, int duration) {
    if (p.isCompleted()) return 100;
    if (duration <= 0) return 0;
    return (int) Math.min(100, (long) p.getMaxPositionSec() * 100 / duration);
}

private static ProgressView toProgress(VideoProgress p, int duration) {
    ProgressView v = new ProgressView();
    v.positionSec = p.getPositionSec();
    v.completed = p.isCompleted();
    v.percent = percentOf(p, duration);
    return v;
}

private static VideoView toView(CourseVideo v, VideoProgress p) {
    VideoView out = new VideoView();
    out.id = v.getId();
    out.courseId = v.getCourseId();
    out.title = v.getTitle();
    out.description = v.getDescription();
    out.sizeBytes = v.getSizeBytes();
    out.durationSec = v.getDurationSec();
    out.uploadedAtMs = v.getUploadedAtMs();
    if (p != null) out.progress = toProgress(p, v.getDurationSec());
    return out;
}
}

```

exception/VideoUploadExceptionAdvice.java (new file)

```

package com.examly.springapp.exception;

import jakarta.servlet.http.HttpServletRequest;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.multipart.MaxUploadSizeExceededException;
import org.springframework.web.multipart.MultipartException;
import org.springframework.web.multipart.support.MissingServletRequestPartException;

/**
 * Gives clear answers for upload problems that Spring raises before our code runs (a file over the size
 * limit,
 * or no file in the request). Runs before GlobalExceptionHandler, which would otherwise answer "500".
 * The body has the same shape as every other error: status, error, message, path.
 */
@RestControllerAdvice
@Order(Ordered.HIGHEST_PRECEDENCE)
public class VideoUploadExceptionAdvice {

    private ResponseEntity<Map<String, Object>> reply(HttpStatus status, String message, HttpServletRequest
req) {
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", message);
        body.put("path", req.getRequestURI());
    }
}

```

```

        return ResponseEntity.status(status).body(body);
    }

    @ExceptionHandler(MaxUploadSizeExceededException.class)
    public ResponseEntity<Map<String, Object>> tooLarge(MaxUploadSizeExceededException ex,
HttpServletRequest req) {
        return reply(HttpStatus.PAYLOAD_TOO_LARGE, "The file is larger than the upload limit set on the
server", req);
    }

    @ExceptionHandler(MissingServletRequestPartException.class)
    public ResponseEntity<Map<String, Object>> missingPart(MissingServletRequestPartException ex,
HttpServletRequest req) {
        return reply(HttpStatus.BAD_REQUEST, "Choose a video file to upload", req);
    }

    @ExceptionHandler(MultipartException.class)
    public ResponseEntity<Map<String, Object>> notMultipart(MultipartException ex, HttpServletRequest req)
{
        return reply(HttpStatus.BAD_REQUEST, "The upload could not be read. Send the video as a form
upload.", req);
    }
}

```

controller/VideoController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.config.RateLimiter;
import com.examly.springapp.dto.VideoDtos.*;
import com.examly.springapp.exception.RateLimitExceededException;
import com.examly.springapp.model.CourseVideo;
import com.examly.springapp.service.AccessService;
import com.examly.springapp.service.VideoService;
import com.examly.springapp.video.RangeParser;
import com.examly.springapp.video.VideoStorageService;
import com.examly.springapp.video.VideoStreamer;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.nio.file.Path;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

/** Lesson videos: upload (admin / owning instructor), watch (staff or enrolled customer) and progress. */
@RestController
@RequestMapping("/api/video")
@Tag(name = "Video", description = "Lesson videos, streaming and watch progress")
public class VideoController {
    private final VideoService service;
    private final VideoStorageService storage;
    private final AccessService access;
    private final RateLimiter limiter;

    @Autowired
    public VideoController(VideoService service, VideoStorageService storage, AccessService access,
RateLimiter limiter) {
        this.service = service;
        this.storage = storage;
        this.access = access;
        this.limiter = limiter;
    }
}

```

```

@Operation(summary = "Upload a video to a course (admin, or the approved instructor who owns it)")
@PostMapping("/course/{courseId}")
public ResponseEntity<VideoView> upload(@PathVariable Long courseId,
                                       @RequestParam("title") String title,
                                       @RequestParam(value = "description", required = false) String
description,
                                       @RequestParam(value = "durationSec", required = false) Integer
durationSec,
                                       @RequestParam("file") MultipartFile file) {
    String key = "video-upload:" + access.currentEmail();
    if (!limiter.tryAcquire(key, 30, 60 * 60_000L)) {
        throw new RateLimitExceededException("Too many uploads. Try again later.",
limiter.retryAfterSeconds(key, 60 * 60_000L));
    }
    return ResponseEntity.status(HttpStatus.CREATED).body(service.upload(courseId, title, description,
durationSec, file));
}

@Operation(summary = "Videos of a course (customers also get their own progress)")
@GetMapping("/course/{courseId}")
public ResponseEntity<List<VideoView>> list(@PathVariable Long courseId) {
    return ResponseEntity.ok(service.list(courseId));
}

@Operation(summary = "Customers' progress on a course's videos (admin or owning instructor)")
@GetMapping("/course/{courseId}/stats")
public ResponseEntity<CourseStats> stats(@PathVariable Long courseId) {
    return ResponseEntity.ok(service.stats(courseId));
}

@Operation(summary = "My courses with video progress and where to resume (customer)")
@GetMapping("/my-learning")
public ResponseEntity<List<LearningItem>> myLearning() {
    return ResponseEntity.ok(service.myLearning());
}

@Operation(summary = "Get a short-lived address for the video player")
@GetMapping("/{videoId}/ticket")
public ResponseEntity<TicketView> ticket(@PathVariable Long videoId) {
    return ResponseEntity.ok(service.ticket(videoId));
}

@Operation(summary = "Save how far I have watched (customer)")
@PutMapping("/{videoId}/progress")
public ResponseEntity<ProgressView> progress(@PathVariable Long videoId, @RequestBody ProgressRequest
body) {
    return ResponseEntity.ok(service.saveProgress(videoId, body));
}

@Operation(summary = "Delete a video (admin or owning instructor)")
@DeleteMapping("/{videoId}")
public ResponseEntity<Void> delete(@PathVariable Long videoId) {
    service.delete(videoId);
    return ResponseEntity.noContent().build();
}

/**
 * The video bytes. Open to the player without a login header, but only with a valid ticket for this
exact video.
 * Supports the Range header so the viewer can jump around without downloading everything first.
 */
@Operation(summary = "Stream a video (needs a ticket from the ticket endpoint)")
@GetMapping("/stream/{videoId}")
public void stream(@PathVariable Long videoId, @RequestParam(value = "t", required = false) String
ticket,
                  HttpServletRequest request, HttpServletResponse response) throws IOException {
    CourseVideo v = service.videoForStream(videoId, ticket);
    Path file = storage.resolve(v.getStoredName());
    if (file == null) {
        response.setStatus(404);
        return;
    }
}

```

```
    }  
    try {  
        VideoStreamer.write(file, v.getContentType(), request.getHeader("Range"), response);  
    } catch (RangeParser.NotSatisfiableException e) {  
        response.setStatus(416);  
        response.setHeader("Content-Range", "bytes */" + file.toFile().length());  
    }  
}
```